Hands-on L2: GitHub Basics

The purpose of this hands-on is to give you practical experience with GitHub as a collaboration and version control platform. You will learn how to create and manage repositories, track changes, work with branches, submit pull requests, and use GitHub's collaboration tools such as Issues, Projects, and Codespaces. By the end of the exercise, you should be comfortable performing the essential tasks needed to manage your own project repository and to contribute to team projects using professional workflows.

Learning goals:

- **Create and manage GitHub repositories**: including adding files, editing content, and tracking changes with commits.
- **Collaborate using GitHub workflows**: branching, pull requests, and code reviews.
- **Organize and track work**: using GitHub Issues and Projects for task management.
- **Develop in the cloud**: experimenting with GitHub Codespaces for browser-based coding and collaboration.

What you will see:

1. Introduction to GitHub
2. Setting Up a GitHub Account
3. Creating a Repository (Web)
4. Exploring Repository Structure
5. Creating & Editing Files in Browser
6. Branches
7. Pull Requests (PRs)
8. Issues
9. Projects (Kanban Boards)
10. Forking a Repository
11. Codespaces (Web Development Environment)
12. Share the Repository

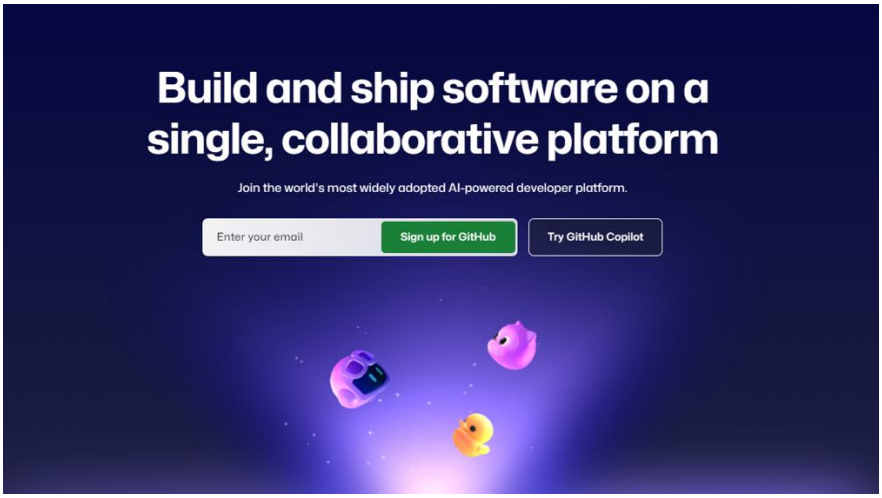
1. Introduction to GitHub

GitHub is a cloud-based platform for hosting code and collaborating on projects. Using GitHub's web interface, you can create repositories, track issues, manage projects, and even edit code directly in the browser without installing anything locally.

Additional References: start-your-GitHub-journey

2. Setting Up a GitHub Account

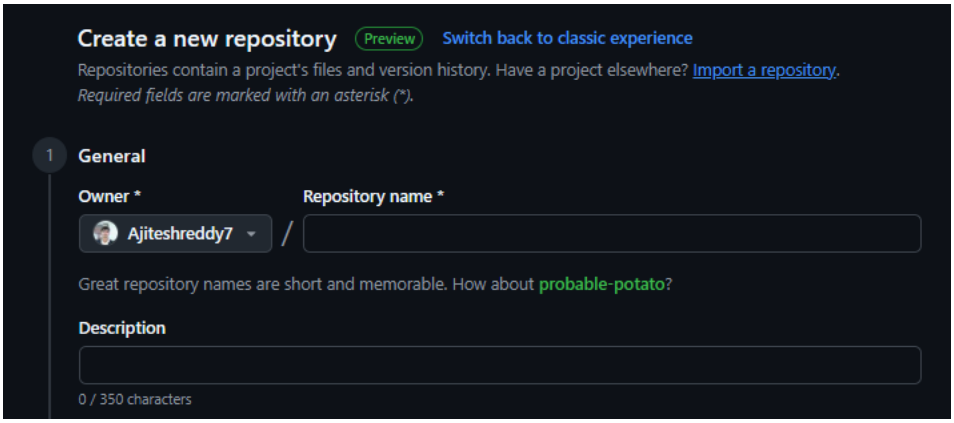
- Go to https://github.com and sign up with your **@charlotte.edu mail**.
- Verify your email.
- Set your profile details (name, picture, bio).



3. Creating a Repository

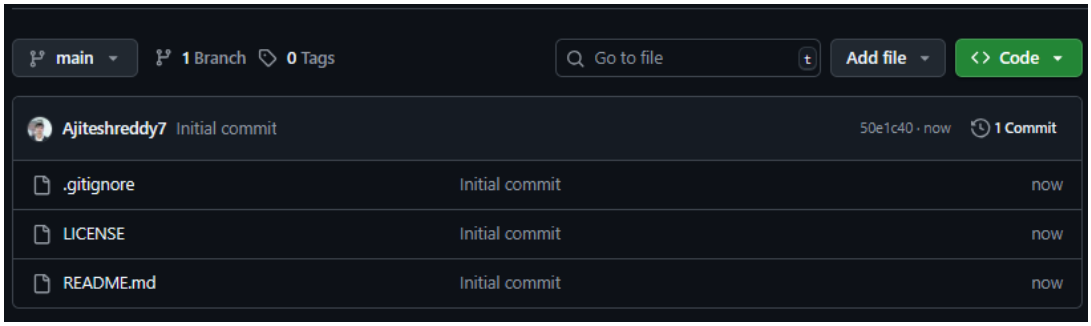
GitHub repository serves as a central hub for all project files, including code, documentation, images, and any other associated assets.

- Click "+" (top-right) → New Repository.
- Fill in repo name, description, and visibility (Public/Private).
- Initialize with a README file.



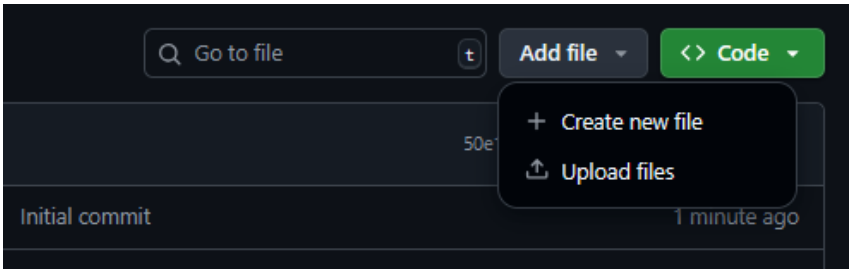
4. Exploring Repository Structure

- Files and folders appear on the main repo page.
- Common files:
  - **README.md** → Introduction and instructions for the project.
  - **.gitignore** → Lists files/folders Git should not track while committing the changes.
  - **LICENSE** → Defines permissions and usage rights for the project.



5. Creating & Editing Files

- Click Add file → Create new file.
- Edit files directly in the browser.
- Use the Preview tab for Markdown.
- Save changes with the Commit changes button.

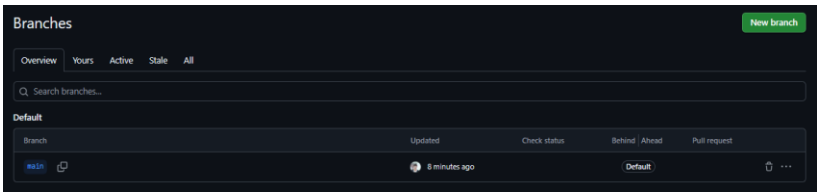


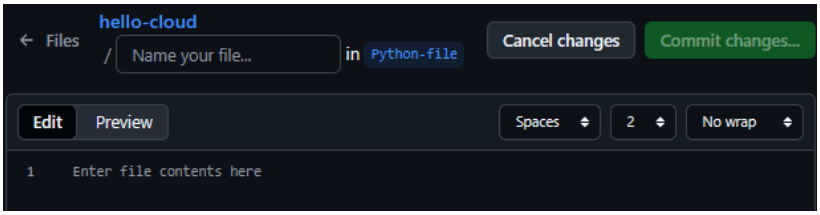
6. Branches

A **branch** is like a separate line of development in your repository. Think of it as a way to work on new features, bug fixes, or experiments **without affecting the main codebase**.

- Click branch dropdown → New Branch.
  - **To-Do:** add a Python file (include .py in file name) with a basic Python code of your liking.

- Commit changes.

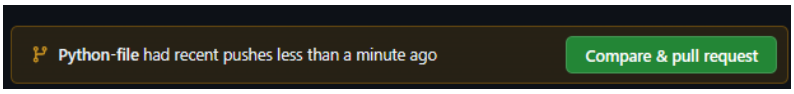


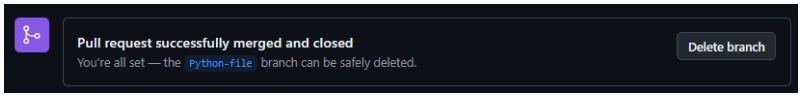


7. Pull Requests (PRs)

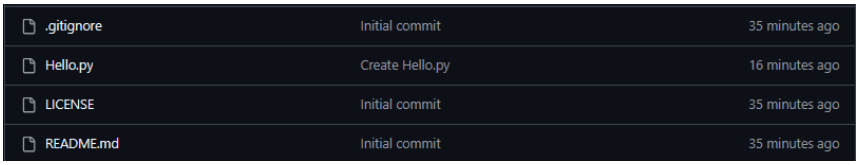
A **Pull Request (PR)** in GitHub lets you tell others that you've pushed changes to a branch and want them reviewed and merged into another branch (often the main branch).

- Go to the Pull Requests tab → New Pull Request.
- Compare your branch with the main branch.
- Add title & description, then submit.
- Teammates can review, comment, and merge.
- **Delete the branch** after you have successfully merged it to main.





After this step, your repo should look something like this:



Also, **edit** the **Readme file** to include your **Name, Student ID, and Email**.

8. Issues

In GitHub, an **issue** is a way to track tasks, bugs, feature requests, or questions within a repository.

TODO:

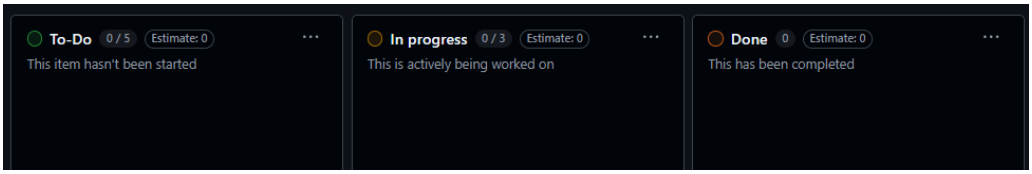
For this part you should edit your python script to introduce an error in the script and log it as an issue.

- Modify the python script, for example the hello world program to as follows: **print("Hello, World!"** and Commit the modified script into the repository. **Note** the missing ")" at the end will cause a **Syntax Error**.
- Issues can be bound under the Issues tab and click on the New Issue button to create the Issue.
- Here open an issue with the title **Syntax Error in python script**.
- In the description section explain what the issue is, Expected vs actual behavior of the issue code, any screenshots or logs.
- On the right sidebar, add label **bug**.
- **Assign the issue to yourself**.
- Click the **Submit new issue** button at the bottom.

9. Projects

GitHub has a **Projects** feature that lets you organize issues, pull requests, and notes on a **Kanban-style board.**

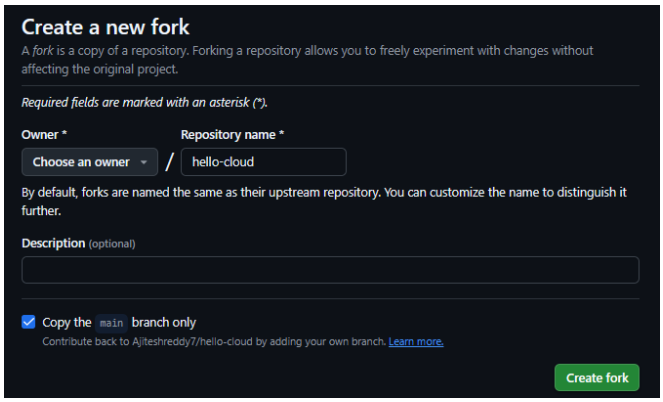
- Go to the Projects tab → New Project.
- Choose Board (Kanban) layout.
- Create columns (To Do, In Progress, Done).
- Add issues/tasks as cards.



10. Forking a Repository

Forking a repository on GitHub means creating your **own copy of someone else's repository** under your account.

- Click the Fork button (top-right).
- Creates a personal copy of someone else's repo.
- Useful for open-source contributions.

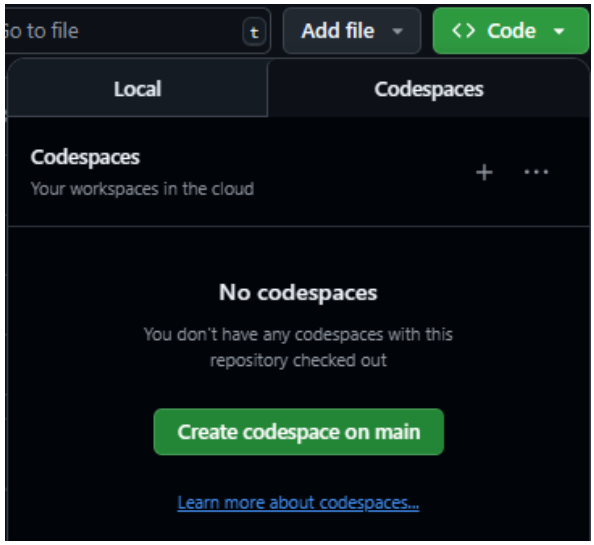


11. Codespaces (Web Development Environment)

GitHub Codespaces is a cloud-based development environment provided by GitHub. It gives you a full-featured Visual Studio Code (VS Code) instance running in the cloud, directly connected to your GitHub repository.

- Click Code → Open with Codespaces → New Codespace.
- Opens a browser-based VS Code environment.

- Allows editing, running, and testing code in the cloud.



12. Share the Repository

Copy its GitHub URL and submit it on Canvas. Make sure that:

- The repository is set to private.
- The course instructor and TAs have been added as collaborators, with at least “read” access.
- The repository contains all required outcomes (from the steps presented above).